

Finding Near Neighbors Through Local Search

Guillermo Ruiz¹, Edgar Chávez²(✉), Mario Graff³, and Eric S. T  llez³

¹ Universidad Michoacana de San Nicol  s de Hidalgo, Morelia, Mexico

`gruiz@dep.fie.umich.mx`

² CICESE, Ensenada, Mexico

`elchavez@cicese.mx`

³ INFOTEC M  xico, Mexico, Mexico

`{mario.graff,eric.tellez}@infotec.com.mx`

Abstract. Proximity searching can be formulated as an optimization problem, being the goal function to find the object minimizing the distance to a given query by traversing a graph with a greedy algorithm. This formulation can be traced back to early formulations defined for vector spaces, and other recent approaches defined for the more general setup of metric spaces.

In this paper we introduce three searching algorithms generalizing to local search other than greedy, and experimentally prove that our approach improves significantly the state of the art. In particular, our contributions have excellent trade-offs among speed, recall and memory usage; making our algorithms suitable for real world applications. As a byproduct, we present an open source implementation of most of the near neighbor search algorithms in the literature.

1 Introduction

The problem of proximity search consists in identifying objects from a collection that are *near* a given query. In spaces with high intrinsic dimension it is difficult to avoid a sequential scan. For those kind of situations, the only practical option is to use *approximate indexes* with recall smaller than one.

A large portion of the literature in proximity searching is devoted to spaces with coordinates with some Minkowsky distance. In this realm, vector quantization[1] is the fastest option; although with low recall in practice. One interesting option is the *Randomized Neighborhood Graph*(RNG)[2] with vertex set the database and random edges in circular cones attached to the objects. With $O(n^2)$ construction cost and $O(n \log n)$ space, the RNG is impractical, and cannot be generalized to general metric spaces; however the ideas are appealing.

Combinatorial Approaches and Rank Cover Trees. In [3], the authors introduced the Range Cover Trees (RCT), where a tree is built using ordered rank for pruning instead of rules derived from distances and the triangle inequality. Node descendants in the tree are obtained using the rank order.

Approximate Proximity Graph. Malkov et al [4] introduced the *Approximate Proximity Graph* (APG), an index with excellent searching times. APG can trade speed and accuracy for memory usage.

The construction is incremental and consists of a simple rule. To insert the j -th element, simply find the (approximate) t -nearest neighbors among the $j - 1$ elements already in the collection and link them to the newly inserted element.

In a subsequent paper [5], the searching is improved, as described below in Algorithm 1.

Algorithm 1. The search algorithm for k nearest neighbors of APG as described in [5]. We use our notation $\mathcal{N}(u)$ to describe the connected vertices of u , see Section 2 for more details.

Name: Search algorithm APG

Input: A transition function $\mathcal{N}$ (neighbors), the database S , and the query q , the number of restarts m

Output: The set res of near neighbors of q .

```

1: Let  $res$  be an empty min-queue of fixed size  $k$ 
2: Let  $candidates$  be an empty min-queue
3: for  $i = 1$  to  $m$  do
4:   Define  $r$  as current distant radius in  $res$ , empty  $res$  defines  $r = \infty$ 
5:   Randomly select  $c \in S \setminus visited$ 
6:   Append  $c$  into visited and  $(d(q, c), c)$  into  $candidates$  and  $res$ 
7:   loop
8:     Let  $(r_b, best)$  be the nearest pair in  $candidates$ 
9:     Remove  $best$  from  $candidates$ 
10:    if  $r_b > r$  then
11:      break loop
12:    end if
13:    for  $u \in \mathcal{N}(best)$  do
14:      if  $u \notin visited$  then
15:        Add  $u$  to  $visited$  and  $(d(q, u), u)$  to  $candidates$  and  $res$ 
16:      end if
17:    end for
18:  end loop
19: end for

```

k-Nearest References (KNR). In some indexes, every node is associated with a set of k nearest references (KNR), where the set of references is a sample of the database. The similarity between items is hinted by a similarity function over the shared neighbors. In this approach there is no navigation and proximity queries are solved using an inverted index. This structural similarity was systematically explored in [6] adding several indexes to the list, like the KNR-Cos that uses the cosine similarity. The experimental evidence suggests that it surpasses the majority of the state of the art KNR indexes, so we will use it in our experimental section.

Contribution. We introduce three new indexes for near neighbor search i) APG* an improvement over APG computing *online* m (the number of restarts), ii) APG*-R that improves APG* limiting the shared state among search steps, and iii) BS, which is a beam search based algorithm.

2 Improving APG

Let S be the database, define $\mathcal{N} : S \rightarrow S^+$ as a transition function describing the connection among items, i.e., $\mathcal{N}(u)$ returns the neighborhood of u . We can think of $\mathcal{N}$ as the edges of a graph over S . Consider the nearest neighbor (nn) problem and let $d_q(u) = d(q, u)$. We want to find $w \in S$ that is the global minimum of d_q and we only have access to $\mathcal{N}(u)$ for a given u . We only have local information. An *optimal* $\mathcal{N}$ has minimum size and allows to find the global minimum using local search. It turns out that the construction of an optimal $\mathcal{N}$ is NP-hard and in some cases, this optimal $\mathcal{N}$ defines the complete graph (equivalent to sequential scan). In order to overcome this, we need to relax our constraints allowing approximate results. For approximate search, it is accepted to locate a good enough local minimum for d_q .

Recall from Section 1 that APG's search is repeated m times to improve the expected recall. As the authors suggest in [5], m must be adjusted for each dataset. A significant overestimation of m would impact both search and construction performances.

Algorithm 2. Beam Search over S and $\mathcal{N}$ for k nearest neighbors.

Name: Beam Search

Input: A transition function $\mathcal{N}$, the database S , the query q , the size of the beam b

Output: The set res of near neighbor of q .

```

1: Let  $res$  be an empty min-queue of fixed size  $k$ 
2: Let  $beam$  be an empty min-queue of fixed size  $b$ 
3: Let  $visited \leftarrow \emptyset$ 
4: Let  $beam \leftarrow \{\}$ 
5: for  $i = 1$  to  $b$  do
6:   Randomly select  $u$  from  $S$ 
7:   Add  $u$  to  $visited$  and  $(d(u, q), u)$  to  $res$  and  $beam$ 
8: end for
9: repeat
10:   Let  $cov* \leftarrow cov(res)$ 
11:   for  $i = 1$  to  $\sigma$  do
12:     Let  $beam* \leftarrow \{\}$  {Fixed sized priority min-queue of size  $b$  as  $beam$ }
13:     for  $c \in beam$  do
14:       for  $u \in \mathcal{N}(c)$  do
15:         if  $u \notin visited$  then
16:           Add  $u$  to  $visited$  and  $(d(q, u), u)$  to  $res$  and  $beam*$ 
17:         end if
18:       end for
19:     end for
20:      $beam \leftarrow beam*$ 
21:   end for
22: until  $cov* = cov(res)$  {Stops when there is no improvement over  $d_q$ , i.e.,  $cov(res)$ }

```

Let $result$ be a min priority queue (min-queue) of fixed size k . If $result$ is under its full capacity then $cov(result)$ equals to the maximum possible distance value; if it is full, then $cov(result)$ is the radius of the furthest item.

The objective of our first search algorithm is to compute m online checking the changes over the global result set, i.e., the covering radius $cov(res)$. Here, we are minimizing d_q , and stopping the algorithm when our guess of d_q cannot be

improved after σ tries. All our new algorithms use the same technique as stop condition. σ should be small, in the range of 2 to 4. The idea is to relax the stop condition, but not to introduce an extra parameter. This search algorithm is used to produce the index APG*.

Our second index is named APG*-R. APG*-R uses random starting points, not the best known in APG and APG*, and also, both *candidates* and *res** are local to each step. This modification helps to escape from a local minima. For lack of space we do not include the full, formal description of the algorithm.

Our final index is BS (produced by Algorithm 2), a novel index using beam search over S and $\mathcal{N}$. As in the previous algorithms, at least σ steps are taken before stopping the search. Hence, if after some steps d_q does not change, then we claim a good local minimum. The main parameter is the size of the beam b , which can be adjusted at any time. Notice that beam search does not need to restart; however, it needs to know b .

3 Experimental Results

All experiments were performed in a 24-core Intel Xeon 2.60 GHz workstation with 256GB of RAM, running CentOS 7, without using the multiprocessing capabilities in the search process. Both, the index and the database were maintained in memory. Our implementations were created in the mono framework, in C#. For the RCT [3] we used the C++ implementation, kindly shared by authors. We fixed our attention in the recall and search speed. Since comparing them by time would be unjust, we measure the time of the search compared with the sequential scan (speedup).

Our benchmarks consist of synthetic databases called RVEC which are random vectors using the L_2 distance. We tested the effect of varying the dimension and database size. First, we used datasets of 16, 32, 64, 128 and 256 dimensions, each one with 10^5 randomly generated items. Next, the performance as the size increases was measured using 16, 32 and 64-dimensional datasets of sizes 3×10^5 , 10^6 , and 3×10^6 items.

The Effect of the Dimension on the Search Performance. On Figure 1, each point on the lines represents the result for a dimension. We show the recall and the speedup for each instance on the same figure. The results are grouped by size of the indexes on the figures (controlled by the parameter t).

Figure 1(a) shows the speedup for the indexes that take from 1 to 10 integers per element of the database. The APG and APG* are very fast because of the small number of distances they have to compute but their poor recall makes them useless. The APG*-R variant shows a big improvement on the recall. The different instances of the BS gain in recall but lose speed when we increase the number of searches. The KNR gets good recalls but is slower than the average of the others. Finally, the RCT has a good overall performance specially for $c = 128$.

The Figure 1(b) contains the speeds for indexes using 10 to 30 integers per element. Because of their small recall, the APG and APG* are very fast. The

APG*-R and BS with $b = 8$ are very similar, one a little faster and the other with better recall. As expected, the other configurations of BS have higher recalls.

Note how the APG and APG* are always very similar, also they need more memory to achieve good recalls. The APG*-R and BS have a great range of action with the option to get recalls very close to one with good speeds.

The Figures 1(c) and 1(d) are very similar. The APG and APG* are not very fast. The APG*-R is faster for the dimension 16, 32, and 64 but the BS with $b = 8$ becomes faster for the big dimension.

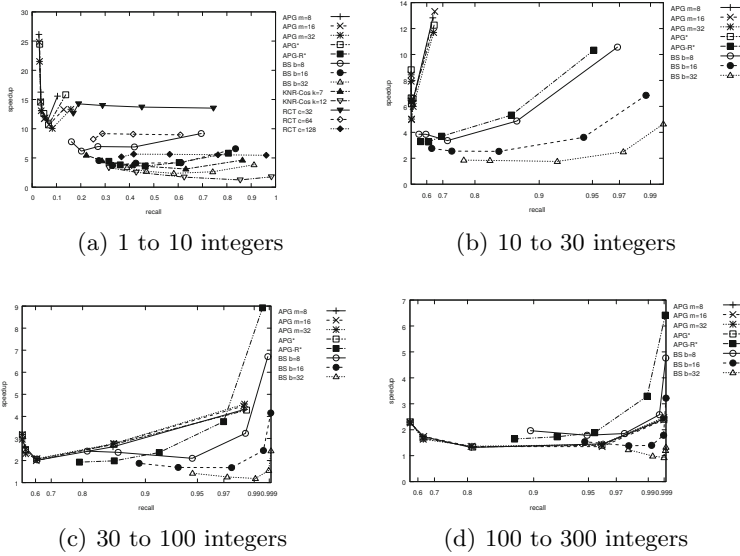


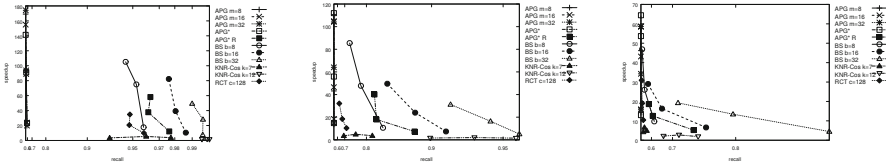
Fig. 1. Recall and speedup comparison in four different classes of memory among our indexes and several state of the art techniques for fixed $n = 10^5$ over several dimensions. Here we show performance for low to medium memory resources (i.e., 1 to 30 integers per item). Each curve corresponds to a different dimension 16, 32, 64, 128, and 256; as a hint to review figures, large dimensions correspond to smaller recalls and scale is exponential.

Scalability. We used the randomly generated database of dimension 16 with sizes 3×10^5 , 10^6 , and 3×10^6 . Each point in a curve is produced by a different database size; speed ups are larger as n increases.

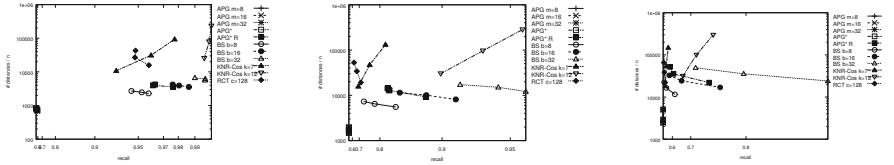
Figure 2(a) shows the recall and the search speed for the different sizes of the database for 16 dimension. The APG and APG* scored a recall below 0.7 on all the sizes even when using 32 searches for the query. As shown in the previous experiments, the APG*-R gets a boost on the recall compared to the APG*. Note how the lines of the BS are practically vertical meaning that the

index gains in speed much more than it loses in the recall. Compare this to the KNR with $k = 7$, its recall is very affected as the size increase. Also, in general the KNR are not very fast. The RCT does not get a recall as good as the BS and is considerable slower. On Figure 2(d) we see the recall and the number of distances for the queries. This general behavior is the same in the next figures.

Figure 2(b) shows the same tendency, the APG*-R being an improvement over the APG's, the BS having very good recalls and speeds, the KNR being slow, and the RCT behind the BS. The Figure 2(e) shows the same but with the distances computed. The same can be said about the Figure 2(c) and 2(f) but the interesting part is to note how the change of dimension affects more the indexes than the size of the database. For 16 dimensions, the performance is maintained for increasing database size.



(a) Speedup / 16dimensions (b) Speedup / 32dimensions (c) Speedup / 64dimensions



(d) Cost / 16 dimensions (e) Cost / 32 dimensions (f) Cost / 64 dimensions

Fig. 2. Speedup as function of the expected recall for our indexes and the state of the art. We use datasets with dimensions 16, 32, and 64. Each dataset has three instances with sizes of 3×10^5 , 10^6 , and 3×10^6 . We also limited memory to be under 8 direct neighbors per item, i.e. 16 undirected neighbors in average.

4 Conclusions

We introduced three near neighbor searching indexes, called APG*, APG*-R and BS. These indexes use an underlying graph and several variants of local search to navigate them. As part of our contribution, we compare our techniques among the current state of the art indexes. Our indexes significantly surpass the performance of the majority of the alternatives, in almost all of our benchmarks.

In the present work we used $\mathcal{N}$ as introduced by APG; however, there exists several open questions about it, such as how to determine its precise parameters online, i.e., without having any a priori knowledge about it.

References

1. Silpa-Anan, C., Hartley, R.: Optimised kd-trees for fast image descriptor matching. In: IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2008, pp. 1–8, June 2008
2. Arya, S., Mount, D.M.: Approximate nearest neighbor queries in fixed dimensions. In: Proceedings of the Fourth Annual ACM/SIGACT-SIAM Symposium on Discrete Algorithms, pp. 271–280, Austin, Texas, 25–27 January 1993 (1993)
3. Houle, M.E., Nett, M.: Rank cover trees for nearest neighbor search. In: Brisaboa, N., Pedreira, O., Zezula, P. (eds.) SISAP 2013. LNCS, vol. 8199, pp. 16–29. Springer, Heidelberg (2013)
4. Malkov, Y., Ponomarenko, A., Logvinov, A., Krylov, V.: Scalable distributed algorithm for approximate nearest neighbor search problem in high dimensional general metric spaces. In: Navarro, G., Pestov, V. (eds.) SISAP 2012. LNCS, vol. 7404, pp. 132–147. Springer, Heidelberg (2012)
5. Malkov, Y., Ponomarenko, A., Logvinov, A., Krylov, V.: Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* **45**, 61–68 (2014)
6. Chávez, E., Graff, M., Navarro, G., Téletz, E.: Near neighbor searching with K nearest references. *Information Systems* **51**, 43–61 (2015)